Week 10A: Distributed Storage Foundations

When Can a System Safely Say “OK”?

A Deceptively Simple Scenario

```
        char new_balance[4096];          // filled by application
logic
        int fd = open("accounts.db", O_WRONLY);
        write(fd, new_balance, 4096);
        close(fd);
        // <-- power fails here
```

After reboot, is `new_balance` on disk?

The answer depends on at least four things the application never explicitly controls:

1. Whether the page cache flushed the write
2. Whether the disk’s own volatile cache committed it
3. Whether the filesystem journaled the metadata
4. Whether any write barrier reached the media

Part 1

The Linux Write I/O Path

What `write()` Actually Does

When your program calls `write(fd, buf, 4096)`:

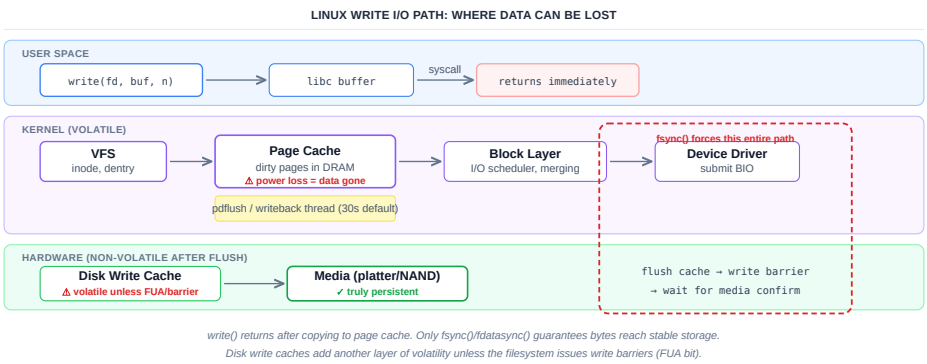
1. Data copies from user buffer into a **page cache page** (kernel DRAM)
2. The page is marked **dirty** in the radix tree

3. `write()` returns — your program continues

That's it. No disk I/O happened. The kernel accepted your bytes into volatile RAM and said "done."

The dirty page will *eventually* be written back by the `writeback` kthread (**default**: every 30 seconds, or when memory pressure triggers it). Until then, a power failure erases the write completely.

The Full Path: User Space → Media



1.00

Key Terms in the I/O Path

Term	What it is
writeback kthread	Kernel thread (`flush-X:Y`) that walks dirty pages and submits them to disk. Runs periodically (~30 s) or when memory pressure triggers it.
pdflush	The *old* name (Linux ≤ 2.6.31) for the writeback thread. Replaced by per-device `bdi-default` / `flush` kthreads. You'll still see the name in older textbooks.
FUA (Force Unit Access)	A bit on a disk I/O command that says "write directly to media, bypass the device's volatile write cache." Avoids a separate cache-flush round trip.
write barrier	A kernel-level I/O ordering guarantee: all writes submitted *before* the barrier must reach stable storage before any write submitted *after* it. Implemented via FLUSH CACHE commands or FUA.

These four pieces control whether data that left the page cache actually reaches non-volatile media — the gap where data loss lives.

Three Layers of Volatility

Layer	What lives there	Survives power loss?
Page cache (kernel DRAM)	Dirty pages waiting for writeback	**No**
Disk write cache (device SRAM/DRAM)	Queued writes the device hasn't committed	**No** (unless battery-backed)
Media (platters / NAND cells)	Bits physically written	**Yes**

The gap between "kernel accepted" and "media persisted" is where data loss lives. Bridging that gap costs latency — and that cost is `fsync`.

Observing the Gap with `strace`

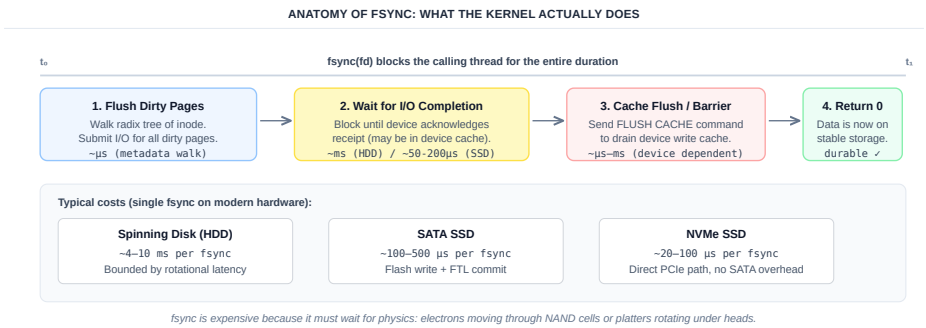
```
# Compile a tiny program that does one write + one fsync
$ cat > /tmp/fsync_demo.c << 'EOF'
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd = open("/tmp/testfile", O_WRONLY|O_CREAT|O_TRUNC,
0644);
    char buf[4096]; memset(buf, 'A', 4096);
    write(fd, buf, 4096);
    fsync(fd);
    close(fd);
}
EOF
$ gcc -o /tmp/fsync_demo /tmp/fsync_demo.c
$ strace -T -e trace=write,fsync /tmp/fsync_demo
write(3, "AAAA"..., 4096)       = 4096 <0.000008>   # 8 µs —
just memcpy
fsync(3)                         = 0    <0.004312>   # 4.3 ms
— waited for disk
```

The 500× latency difference between `write` and `fsync` is not overhead — it's the difference between copying to RAM and waiting for physics.

On an NVMe SSD, `fsync` drops to ~50 µs. On a spinning disk, it's 4–10 ms (one rotation). Either way, it dominates the write-path cost.

> ⚠️ **VM caveat**: In VirtualBox/VMware, `fsync` may appear as fast as `write` — the hypervisor intercepts the flush command and returns immediately without hitting real media. Discuss: what does this mean for durability guarantees inside a VM?

What `fsync` Forces the Kernel to Do



1.00

Why flush *after* I/O completion? Step 2 waits for the device controller to accept the data — but that data may sit in the device's volatile DRAM/SRAM cache. Step 3 sends a FLUSH CACHE command (or uses FUA) to force the device to commit from its volatile cache to the actual NAND/platter. Without Step 3, a power failure after Step 2 can still lose data that the device "acknowledged."

`fsync` vs. `fdatasync` vs. `O_SYNC`

```
        fsync(fd);       // flush data + metadata (inode mtime, size)
        fdatasync(fd);   // flush data + only metadata needed for read correctness
        // O_SYNC flag:  every write() behaves like write + fsync
        int fd = open("f", O_WRONLY | O_SYNC);  // expensive per-write durability
```

Variant	Flushes data?	Flushes metadata?	Use case
`fsync`	✓	all	Journaling, WAL
`fdatasync`	✓	size/blocks only	Append-only logs (skip mtime)
`O_SYNC`	per write	per write	Rare — too expensive for most apps

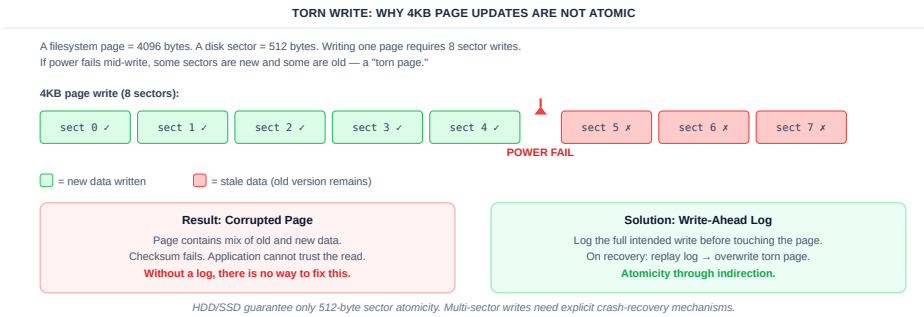
> **`O_DIRECT`** is *not* a durability mechanism — it bypasses the page cache so the application manages its own buffer pool (used by databases like MySQL InnoDB). But `O_DIRECT` alone does **not** guarantee data reaches media. You still need `fsync` or `O_SYNC` on top of `O_DIRECT` for crash safety.

Choosing the right call is a real engineering decision — not a textbook footnote.

Part 2

Why Update-in-Place Breaks Under Crash

The Torn Write Problem



1.00

Sector Atomicity: The Hardware Contract

The only atomicity guarantee hardware gives you:

> **A single sector write is atomic.** Either all bytes in that sector land, or none do.

Traditional drives use 512-byte sectors; modern drives increasingly use 4096-byte physical sectors (4Kn). But a 64KB database page still spans 16 sectors (at 4K) or 128 sectors (at 512B). The hardware makes no promise about multi-sector atomicity.

So if you update data in-place (overwriting the original location), any crash mid-write leaves a page that is neither the old value nor the new value. Without external help, you cannot recover.

Part 3

Write-Ahead Logging as the First Solution

WAL in One Principle

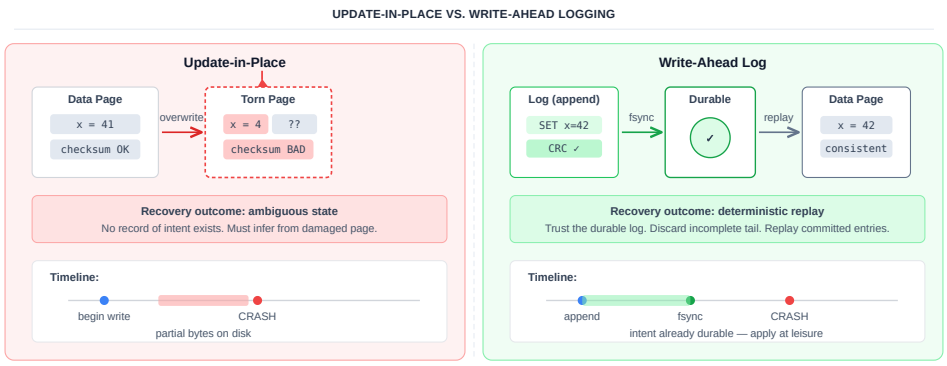
> **Record the intent on stable storage *before* modifying the main data structure.**

For `PUT(x, 42)`:

1. Append a log record: `SET x=42` + CRC
2. `fsync` the log file — this is the durability point
3. Return success to the caller
4. Later (asynchronously), apply the update to the data page

If we crash between steps 3 and 4, recovery replays the log. If we crash before step 2 completes, the incomplete record is discarded — the old value of `x` remains correct.

Why This Works: Separation of Concerns



WAL separates the moment of commitment (log fsync) from the structural update, making crash recovery deterministic.

0.82

Left (update-in-place): overwrite the data page directly → crash mid-write creates a torn page with no recovery path. The old value is destroyed before the new value is complete.

Right (WAL): write the *intent* to a separate log first, `fsync` the log, then apply later. A crash before fsync → discard incomplete tail (old value safe). A crash after fsync → replay the log (new value recovered). The data page is never the single point of truth.

WAL in Real Systems You Already Know

ext4 journal (metadata journaling by default):

```
$ dumpe2fs /dev/sda1 | grep "Journal size"
Journal size: 128M
# Every metadata change is logged before the inode table is modified
```

PostgreSQL WAL:

```
$ ls /var/lib/postgresql/14/main/pg_wal/
000000010000000000000001   # 16MB segment files
```

```
# Every INSERT/UPDATE is logged before heap pages change
```

Redis AOF (RESP protocol, one token per line):

```
$ cat appendonly.aof
*3          # array of 3 elements
$3          # next string is 3 bytes
SET
$1
x
$2
42
```

The pattern is universal. The trade-off is always the same: one extra fsync on the log before you can proceed.

Recovery Is Simple Because the Log Is the Truth

After a crash, the recovery algorithm asks one question:

> **Which log records have a valid CRC and are fully on stable storage?**

Then:

- Committed records → replay them (redo)
- Incomplete tail → discard (it was never acknowledged)
- Main data structure → rebuild from last checkpoint + log

This makes crash recovery **deterministic** — no guessing, no heuristics, no "check every page for corruption." The log defines what happened.

The Log Cannot Grow Forever

An append-only log has three escalating costs:

1. **Recovery time** — replaying 10M records on restart is slow
2. **Disk space** — old entries for overwritten keys waste storage
3. **Read amplification** — scanning the log to find current values is O(n)

Every system that uses WAL eventually needs a cleanup mechanism:

- **Checkpoint / snapshot**: capture consistent state, truncate the log prefix
- **Compaction**: merge old entries, keep only live versions

- **Rewrite**: create a fresh, smaller log (Redis AOF rewrite)

We'll see these concretely in 10B. For now: cleanup is not optional — it's the price of fast appends.

Part 3.5

Redis: WAL in Practice

Why Redis Is Our WAL Case Study

Redis is an **in-memory data structure server** — all data lives in RAM.

It has two optional persistence mechanisms, both built on the principles we just covered:

Mechanism	Principle	What it does
AOF (Append-Only File)	WAL	Appends every write command to a log file
RDB (Redis Database)	Checkpoint / Snapshot	Periodically dumps the full in-memory state to disk

We are not doing a full Redis lecture here — that is Week 10B. But Redis gives us the **simplest concrete WAL** to work with in Lab 10A: the log entry *is* the command itself, and the `fsync` policy is a single config knob.

Redis AOF: The Simplest Possible WAL

When a client sends `SET mykey myvalue`, Redis appends this to the AOF file:

```
*3          ← array of 3 elements
$3          ← next string is 3 bytes
SET
$5
mykey
$7
myvalue
```

This is the WAL principle applied in the most direct way:

1. **Record of intent** = the command itself (in RESP text format)
2. **Durability point** = `fsync` on the AOF file (timing depends on policy)

3. **Recovery** = replay the AOF against an empty Redis instance

You can `cat` the AOF file and read it. You can `strace` the Redis process and count `fsync` calls. In Lab 10A, you will do both.

`appendfsync`: The fsync Policy Is the Durability Contract

Redis lets you choose *when* to call `fsync` on the AOF — and this choice changes what `OK` means:

Policy	When Redis calls `fsync`	What `OK` means to the client
`always`	After every command	This write reached stable storage
`everysec`	~Once per second (group flush)	This write is *probably* safe; up to ~1 s may be lost
`no`	Never (OS writeback decides)	This write is in kernel RAM, not durably safe

This is the same fsync decision from Part 1, made into a product feature.

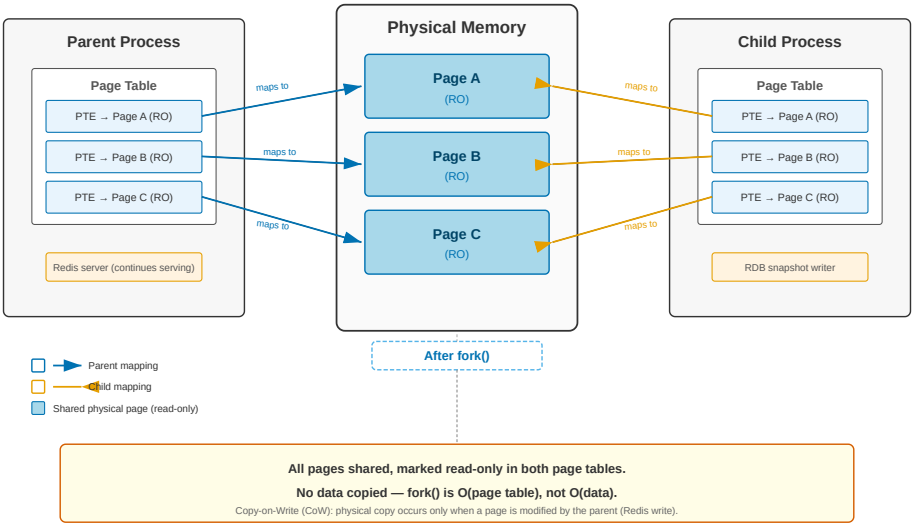
- `always` = per-write `fsync` (like `fsync_per_write.c` in Lab 10A Part 0)
- `everysec` = batched `fsync` (like the batch mode)
- `no` = rely on the writeback kthread (~30 s)

In Lab 10A you will `strace` Redis in `always` and `everysec` modes and **count the fsync calls yourself**.

Redis RDB Snapshot: Step 1 — fork()

The AOF log grows forever. Redis also supports periodic **RDB snapshots**.

When Redis calls `fork()`, the kernel does **not** copy any data pages:



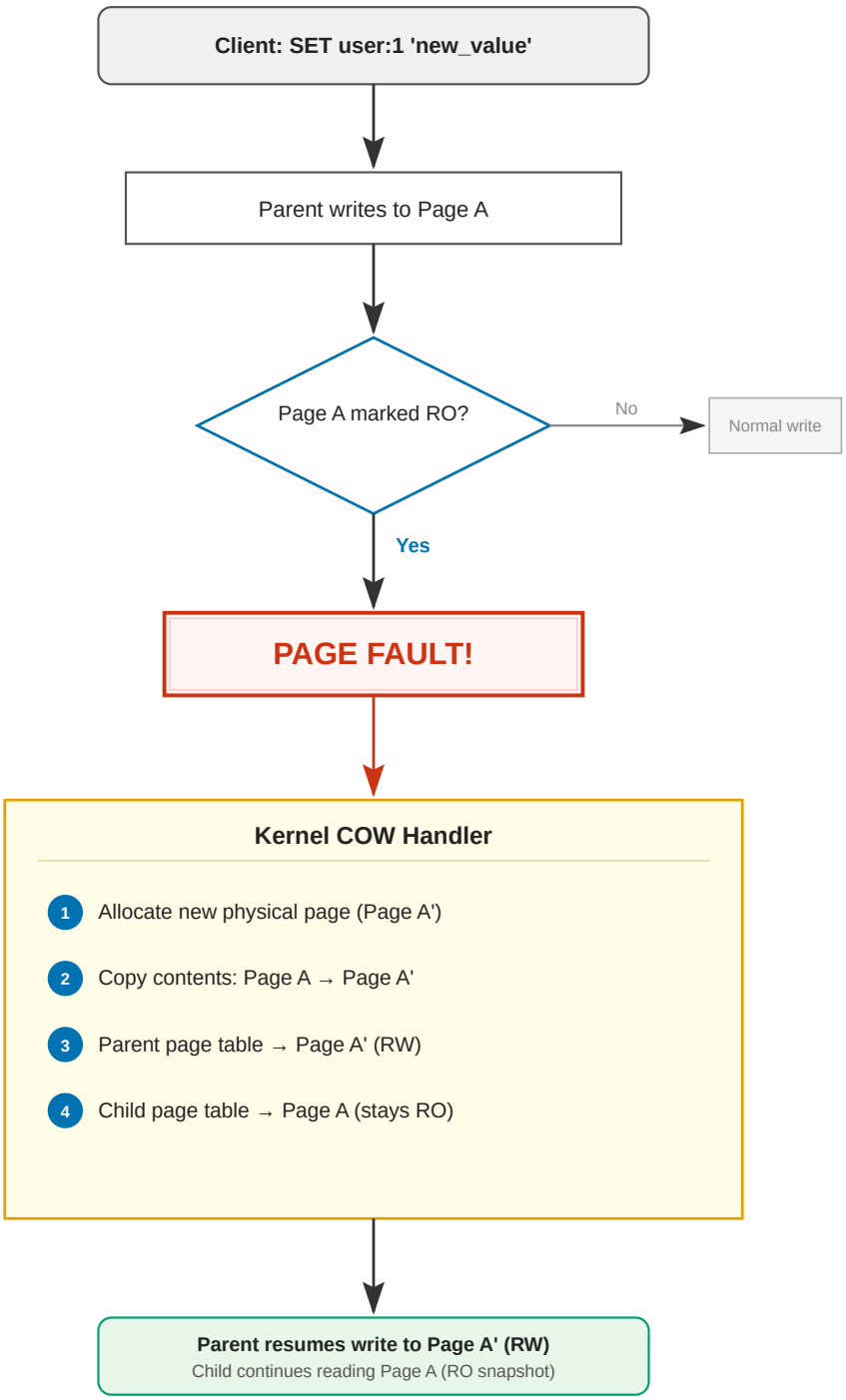
0.90

The child's job: iterate over all shared pages, serialize every key-value pair, write the `.rdb` file to disk. The child **only reads** — it never triggers a COW fault.

Redis RDB Snapshot: Step 2 — Parent Writes Trigger COW

The parent keeps serving all client commands — including writes.

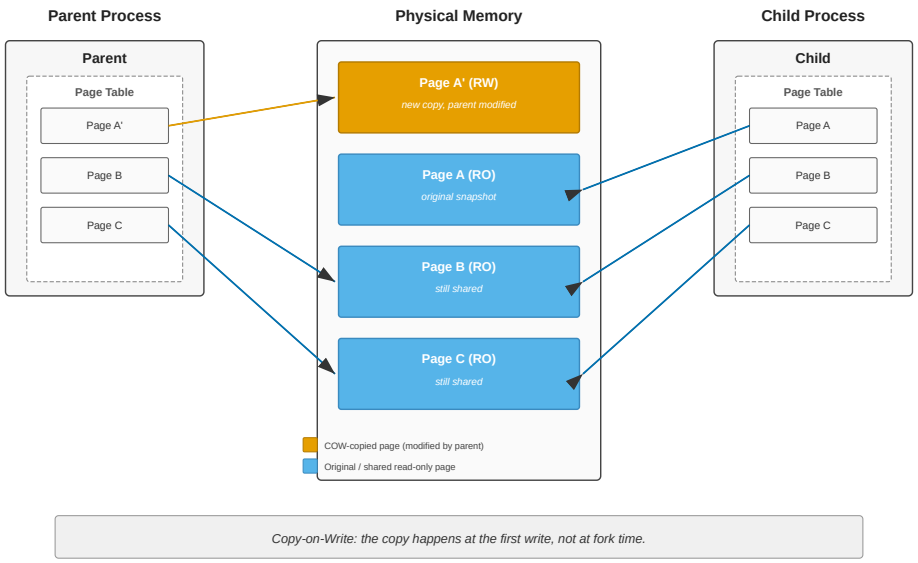
But when the parent modifies a page, it hits the read-only protection set by `fork()`:



0.6

Redis RDB Snapshot: Step 2 — Parent Writes Trigger COW

After the fault:



0.90

This is the Copy-on-Write: the copy happens at the moment of the first write, not at fork time.

Redis RDB Snapshot: Why Memory Can Approach 2×

Every page the parent modifies during the snapshot **gets copied once**:

1.00

- **Light write load** during snapshot → few pages modified → small COW cost
- **Heavy write load** during snapshot → most pages modified → RSS ≈ 2×

Redis reports this as `rdb_last_cow_size` — the total bytes copied by COW during the last snapshot. In Lab 10A Part 4, you will trigger `BGSAVE` under write load and watch this number grow.

Redis RDB Snapshot: Key Properties

Question	Answer
Does Redis become read-only during snapshot?	**No.** Parent keeps serving all commands (reads and writes)
Who triggers the COW copies?	**The parent**, when it writes to a shared page
Does the child trigger COW?	**No.** The child only reads pages to serialize them
Is the snapshot consistent?	**Yes.** The child sees the memory state at the instant of `fork()`. Parent's later writes are invisible to the child — they go to copied pages
What is the worst-case memory cost?	**~2×** RSS if every page is modified during the snapshot
When is this cost acceptable?	When snapshots are infrequent, or write load is low during the snapshot window

> This is **background maintenance** (Part 7 preview): fast in-memory writes create deferred work. The deferred cost shows up as memory pressure and COW page faults during the snapshot.

Part 4

From One Machine to Several

Local Durability Is Not Enough

Everything so far survives process crashes and power failures on one machine. But what about:

- Disk failure (bad sectors, controller death)
- Motherboard failure

- Datacenter fire

One `fsync` on one disk is one copy. If that machine is gone, the data is gone. Replication gives us survival beyond a single point of failure — but at significant cost in complexity.

The Core Design Choice in Replication

Every replicated system faces one question:

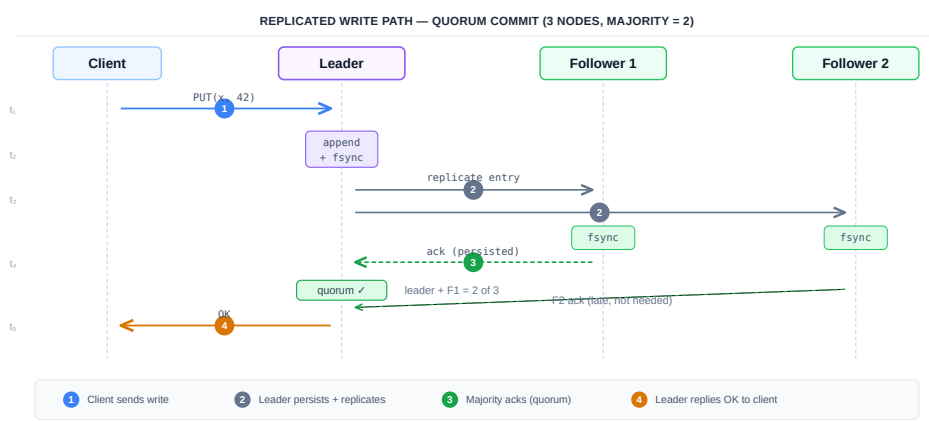
> **After which event do we tell the client "OK"?**

Possible answers:

Reply after…	What we buy	What can still go wrong
Leader RAM only	lowest latency	crash = data lost
Leader `fsync`	survives leader crash	leader disk dies = data lost
One follower ack	survives leader loss	follower might not persist either
Majority fsync	survives any single failure	requires network round-trips

The reply point **is** the safety policy. Everything else is implementation detail.

Replicated Write Path: Quorum Commit



1.00

Why a Majority?

With 3 nodes, any group of 2 overlaps any other group of 2 by at least one node.

That overlap is the key invariant:

- A committed write exists on ≥2 nodes
- A new leader election requires ≥2 votes
- So the new leader **must** contact at least one node that has the committed write

No single node failure can erase a committed entry. No leadership change can forget it. This is the same quorum logic from Week 7 (Raft), now applied to storage safety.

Part 5

Failures That Define the System

Three Scenarios That Break Naive Intuitions

What happened	Is the write safe?	Why
Leader appended to RAM, crashed before `fsync` or replication	**No**	Volatile on one machine — never persisted anywhere
Leader did `fsync` locally, crashed before majority ack	**Depends**	One durable copy exists, but a new leader elected without it may diverge
Entry reached majority, but the client timed out	**Yes**	The write is committed — the reply was lost, not the data

Scenario 3 is the subtle one: the data **is** safe, but the client doesn't know it. **"I didn't see OK" does not mean "the write didn't happen."** This is why storage clients need idempotency keys or request IDs.

The Timeout Ambiguity Problem

When a client times out after `PUT(x, 42)`:

```
Client                    Leader
  |--- PUT(x,42) --------> |
```

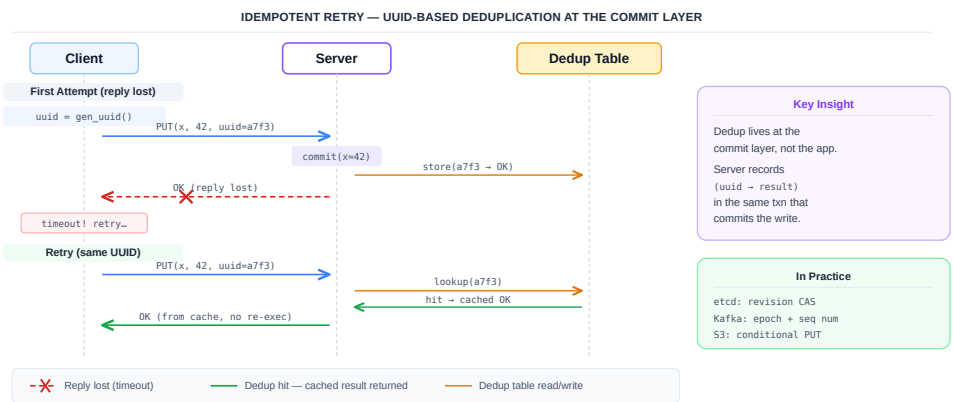
```
|                          |-- fsync, replicate, commit
|                          |
|<-- [OK lost/delayed] --|
|                          |
[timeout!]
```

The client cannot distinguish:

- Nothing happened (safe to retry)
- Write committed, reply lost (retry would be a no-op — fine)
- Write partially replicated (retry might create a duplicate)

Solutions: **request IDs**, **conditional writes** (CAS), or **exactly-once semantics** at the storage layer. This is not an edge case — it's the normal failure mode of distributed writes.

Making Retries Safe: Idempotent Writes



1.00

The Mechanism in Four Lines

```
def handle_put(key, value, uuid):
    if uuid in dedup_table:          # O(1) lookup
        return dedup_table[uuid]     # cached result, no re-execution
    result = commit(key, value)      # actual write + fsync
    dedup_table[uuid] = result       # same transaction
    return result
```

The critical detail: the `dedup_table[uuid] = result` write is **inside the same transaction** that commits the data. If the commit fails, no dedup entry exists, so retry correctly re-executes. If the commit succeeds, the dedup entry guarantees every

subsequent retry returns the same result without a second write.

This is why it's an OS/infrastructure concern, not an application concern — the dedup table lives at the storage commit layer, not in your business logic.

> Real-world TTL: dedup entries can't live forever. Typical retention is 5–15 minutes (Kafka) or tied to a lease/session (etcd). After expiry, the client must treat the operation as unknown — back to the timeout problem.

Part 6

CAP: A Consequence of Partition

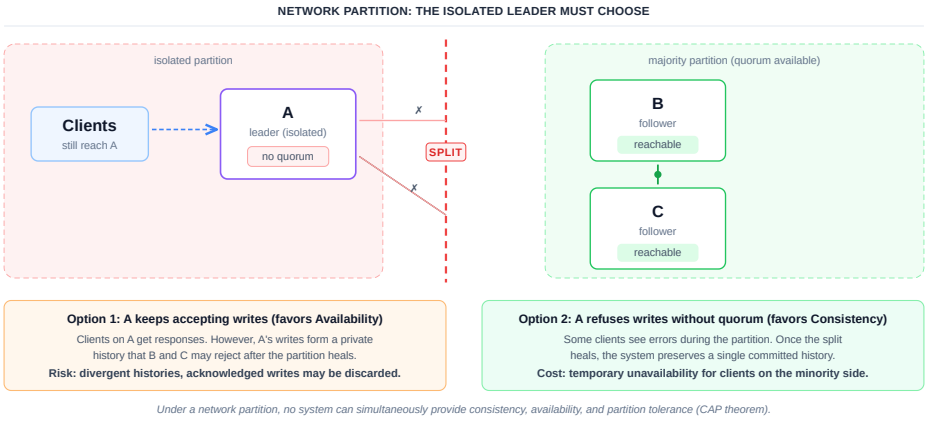
Three Properties, Pick Two (During a Partition)

Property	Definition	Concrete meaning
C — Consistency	Every successful read sees the latest committed write	After `PUT(x,42)` returns OK, no client ever reads `x=41`
A — Availability	Every request to a non-failed node gets a response	No timeouts, no "service unavailable" errors
P — Partition tolerance	The system keeps operating despite network splits	Two halves of the cluster can't talk, but each half tries to serve

Brewer's conjecture (2000, proved by Gilbert & Lynch 2002): during a network partition, you can have at most two of three.

But partitions **will** happen (cables cut, switches fail, BGP flaps). So the real choice is always **CP vs. AP** — "CA" requires a network that never fails.

What Does the Choice Look Like?



1.00

CAP Is a Constraint, Not a Category

Common mistakes:

- "Redis is AP" — **wrong**. Redis with `WAIT` + Sentinel can act CP-ish.
- "etcd is CP" — **true**, but only because it refuses writes without quorum.
- "We need to choose CA" — **impossible** under real network conditions.

CAP tells you what cannot coexist during a partition. It says nothing about:

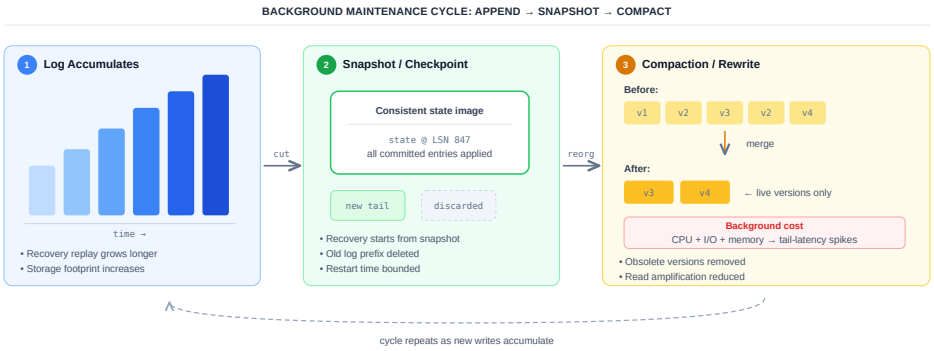
- Steady-state latency
- Throughput
- Operational complexity
- Recovery time after partition heals

Use it as a reasoning tool for partition behavior, not as a product classification.

Part 7

Background Maintenance

The Append Path Is Fast Because Cleanup Is Deferred



1.00

Why This Shows Up as Tail Latency

Background cleanup (compaction, snapshot, checkpoint) competes with foreground writes for:

- **CPU** — scanning and merging records
- **Memory** — holding old and new structures simultaneously
- **I/O bandwidth** — writing compacted output to disk

The result is **tail-latency spikes**, not average slowdown. P99 latency might be 10× P50 during compaction.

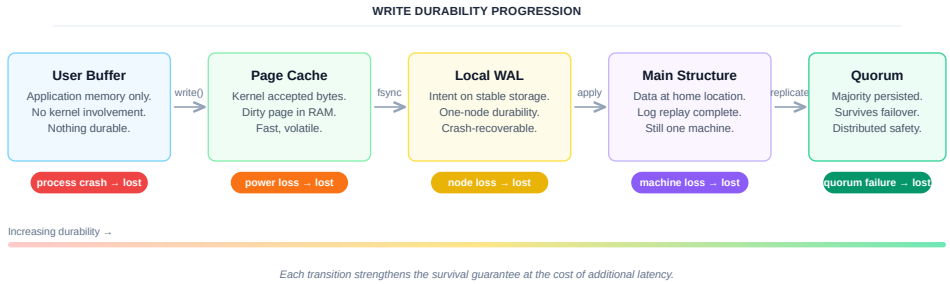
We already saw this concretely in Part 3.5: Redis's `BGSAVE` uses `fork()` + COW to snapshot without blocking the foreground. Under write-heavy load, COW page faults compete for memory and I/O — that's background maintenance cost showing up as resource contention.

Part 8

Synthesis

The Five States of a Write

Now that we've covered the page cache, WAL, and quorum replication, we can see the full progression from "data in user memory" to "data safe from any single failure":



1.00

Each transition adds durability but costs latency. The choice of *where to reply "OK"* defines the system's safety guarantee — and that's the central design decision of every storage system we'll study.

The Three Ingredients of a Durable Distributed Write

Every durable distributed storage system, regardless of implementation, combines:

1. **Record of intent** — a log entry capturing what should happen (WAL, AOF, Raft log)
2. **Durability point** — an `fsync` (or equivalent) that makes the intent survive power loss
3. **Survival rule** — a replication policy that makes the intent survive machine loss (quorum, chain, etc.)

Everything else — caching, batching, compaction, snapshots, read replicas — is optimization or cleanup around these three.

Looking Ahead to 10B

You now know Redis's WAL (AOF) and snapshot (RDB) mechanisms. Week 10B asks the deeper question: **why did Redis make these choices, and what are the alternatives?**

Next session compares three design families:

- **Redis** — RAM-first design, single-threaded event loop, async replication, failover trade-offs
- **etcd** — correctness-first design, Raft consensus, quorum fsync before `OK`, MVCC revisions
- **LSM engines** — write-scalability-first design, sequential appends, multi-level compaction

Same three ingredients (intent record + durability point + survival rule), very different trade-offs.

Lab 10A: What You Will Measure

Part	What you do	What you observe
0	Compile C programs, run `strace`	The raw `write()` vs `fsync` latency gap
1	`strace` the Redis process	fsync call count under `always` vs `everysec`
2	`redis-benchmark` three modes	Throughput cost of durability
3	Kill Redis mid-write (3 trials each)	How many acknowledged writes survive a crash
4	Trigger `BGSAVE` under write load	`rdb_last_cow_size` and RSS growth from fork/COW
5	Design your own experiment	Hypothesis → measurement → analysis

You will produce **figures**, not just numbers. The lab exercises observability and measurability — the course's core theme.